```
// B-Babilônia - /api/criar-pagamento
// Calcula as taxas, guarda-as no pedido e abre o pagamento (cartão ou cripto).
const SUPABASE_URL = 'https://dsfoqoeawvjafypycldz.supabase.co';
const TAXA_PLATAFORMA = Number(process.env.TAXA_PLATAFORMA || 0.05); // 5%
const MZN_POR_USD = Number(process.env.MZN_POR_USD || 64);           // ajuste quando
quiser

function deUsd(valor, moeda) {
  const m = String(moeda).toLowerCase();
  if (m === 'mzn') return valor * MZN_POR_USD;
  return valor;
}

// Moedas sem decimais no Stripe
const SEM_DECIMAIS = ['jpy', 'krw', 'vnd', 'clp'];
// Moedas que o Stripe cobra diretamente (as outras são convertidas para USD)
const STRIPE_OK = ['usd', 'eur', 'gbp', 'brl', 'zar', 'cad', 'aud', 'chf'];

function paraUsd(valor, moeda) {
  const m = String(moeda).toLowerCase();
  if (m === 'usd') return valor;
  if (m === 'mzn') return valor / MZN_POR_USD;
  return valor; // outras moedas: ajuste aqui se precisar
}

async function sbFetch(path, opts) {
  const key = process.env.SUPABASE_SERVICE_ROLE_KEY;
  const r = await fetch(SUPABASE_URL + '/rest/v1/' + path, Object.assign({}, opts, {
    headers: Object.assign({
      apikey: key,
      Authorization: 'Bearer ' + key,
      'Content-Type': 'application/json',
      Prefer: 'return=representation'
    }, (opts && opts.headers) || {})
  }));
  const t = await r.text();
  if (!r.ok) throw new Error('Supabase: ' + t);
  return t ? JSON.parse(t) : null;
}

module.exports = async function handler(req, res) {
  if (req.method !== 'POST') return res.status(405).json({ erro: 'Método inválido'
});
  try {
    const body = typeof req.body === 'string' ? JSON.parse(req.body) : (req.body ||
{});
```

```
    const { order_id, metodo, moeda_cripto, acao } = body;
    if (!order_id) return res.status(400).json({ erro: 'Falta o pedido' });
    if (!process.env.SUPABASE_SERVICE_ROLE_KEY) {
      return res.status(500).json({ erro: 'Falta SUPABASE_SERVICE_ROLE_KEY na Vercel'
});
    }

    const rows = await sbFetch('orders?id=eq.' + encodeURIComponent(order_id) +
'&select=*');
    const o = rows && rows[0];
    if (!o) return res.status(404).json({ erro: 'Pedido não encontrado' });
    if (o.payment_status === 'pago') return res.status(400).json({ erro: 'Pedido já
pago' });

    const moeda = o.currency || 'MZN';
    const subtotal = Number(o.subtotal);
    const taxa_plataforma = Math.round(subtotal * TAXA_PLATAFORMA * 100) / 100;
    // Taxa de cripto: depende da moeda/rede escolhida (onde a troca é feita), vem da
tabela taxas_cripto
    let taxa_cambio = 0;
    if (metodo === 'cripto') {
      if (!moeda_cripto) return res.status(400).json({ erro: 'Escolha a moeda cripto'
});
      const tc = await sbFetch('taxas_cripto?codigo=eq.' +
encodeURIComponent(moeda_cripto) + '&ativo=eq.true&select=*');
      const t = tc && tc[0];
      if (!t) return res.status(400).json({ erro: 'Moeda cripto indisponível' });
      taxa_cambio = Math.round((subtotal * Number(t.percentual) / 100 +
deUsd(Number(t.fixo_usd), moeda)) * 100) / 100;
    }
    const total = Math.round((subtotal + taxa_plataforma + taxa_cambio) * 100) / 100;

    await sbFetch('orders?id=eq.' + encodeURIComponent(order_id), {
      method: 'PATCH',
      body: JSON.stringify({
        total,
        platform_fee: taxa_plataforma,
        fx_fee: taxa_cambio,
        supplier_amount: subtotal,
        payment_method: metodo
      })
    });

    const base = { subtotal, moeda, taxa_plataforma, taxa_cambio, total };
    const origem = req.headers.origin || ('https://' + req.headers.host);

    if (metodo === 'cripto') {
      if (acao === 'cotar') return res.json(base); // só mostra as taxas, ainda não
cria o pagamento
      if (!process.env.NOWPAYMENTS_API_KEY) {
```

```
        return res.status(500).json({ erro: 'Pagamento por cripto ainda não
configurado' });
      }
      const r = await fetch('https://api.nowpayments.io/v1/invoice', {
        method: 'POST',
        headers: { 'x-api-key': process.env.NOWPAYMENTS_API_KEY, 'Content-Type':
'application/json' },
        body: JSON.stringify({
          price_amount: Math.round(paraUsd(total, moeda) * 100) / 100,
          price_currency: 'usd',
          pay_currency: moeda_cripto,
          is_fixed_rate: true,
          is_fee_paid_by_user: false,
          order_id,
          order_description: 'Pedido B-Babilônia',
          ipn_callback_url: origem + '/api/webhook-cripto',
          success_url: origem + '/pagamento-ok.html?pedido=' + order_id,
          cancel_url: origem + '/checkout.html'
        })
      });
      const d = await r.json();
      if (!r.ok || !d.invoice_url) return res.status(502).json({ erro: 'Gateway
cripto: ' + (d.message || 'erro') });
      await sbFetch('orders?id=eq.' + encodeURIComponent(order_id), {
        method: 'PATCH', body: JSON.stringify({ payment_ref: String(d.id) })
      });
      return res.json(Object.assign(base, { url: d.invoice_url }));
    }

    // Cartão (Stripe)
    if (!process.env.STRIPE_SECRET_KEY || !process.env.STRIPE_PUBLISHABLE_KEY) {
      return res.status(500).json({ erro: 'Pagamento por cartão ainda não
configurado' });
    }
    let moedaStripe = String(moeda).toLowerCase();
    let valorStripe = total;
    if (!STRIPE_OK.includes(moedaStripe)) {
      valorStripe = paraUsd(total, moeda);
      moedaStripe = 'usd';
    }
    const minor = SEM_DECIMAIS.includes(moedaStripe) ? Math.round(valorStripe) :
Math.round(valorStripe * 100);
    const form = new URLSearchParams();
    form.append('amount', String(minor));
    form.append('currency', moedaStripe);
    form.append('automatic_payment_methods[enabled]', 'true');
    form.append('metadata[order_id]', order_id);
    form.append('receipt_email', o.customer_email || '');
    const r = await fetch('https://api.stripe.com/v1/payment_intents', {
      method: 'POST',
```

```
      headers: {
        Authorization: 'Bearer ' + process.env.STRIPE_SECRET_KEY,
        'Content-Type': 'application/x-www-form-urlencoded'
      },
      body: form
    });
    const d = await r.json();
    if (!r.ok) return res.status(502).json({ erro: 'Stripe: ' + ((d.error &&
d.error.message) || 'erro') });
    await sbFetch('orders?id=eq.' + encodeURIComponent(order_id), {
      method: 'PATCH', body: JSON.stringify({ payment_ref: d.id })
    });
    return res.json(Object.assign(base, {
      client_secret: d.client_secret,
      publishable_key: process.env.STRIPE_PUBLISHABLE_KEY
    }));
  } catch (e) {
    return res.status(500).json({ erro: e.message });
  }
};
```